# IAM-10: Templated Policy-Group Assignments

> **Series:** IAM — IAM Administration | **Notebook:** 10 of 12 | **Created:** February 2026 | **Last Updated:** 11/10/2026

## Overview

Parameterized policies eliminate the need to duplicate policies for every team, bucket, or security context. Instead of creating 20 nearly-identical policies for 20 teams, you create **one policy template** with `${bindParam:...}` placeholders and supply different values when binding the policy to each group.

This notebook walks you through the complete lifecycle: template syntax, UI creation, IAM API binding, Monaco automation, common patterns, DQL audit queries, and troubleshooting.

---

## Table of Contents


1. Why Policy Templates?
2. Template Syntax Deep Dive
3. Creating a Parameterized Policy via UI
4. Binding with Parameters via IAM API
5. Monaco YAML for Policy-as-Code
6. Common Template Patterns
7. DQL Audit Queries
8. Troubleshooting Templated Policies


---

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with Gen3 IAM enabled |

| Requirement | Details |
|---|---|
| **Permissions** | `account-user-management` to create/modify policies and bindings |
| **API Access** | OAuth client (bearer token) with IAM management scopes — the Account Management API is secured by OAuth 2.0 tokens |
| **Prior Knowledge** | **IAM-04: Policy Authoring and Management** (policy syntax, conditions) |
| **Optional** | Monaco CLI installed (for policy-as-code section) |

# 1. Why Policy Templates?

## The Problem: Policy Sprawl

Without templates, every team that needs scoped data access requires its own dedicated policy. For an organization with 15 teams, that means:

- **15 nearly-identical policies** to create
- **15 policies to maintain** when permissions change
- **15 policies to audit** during compliance reviews

## The Solution: Parameterized Policies

With templates, you create **one policy** with `${bindParam:...}` placeholders and create **15 bindings** with different parameter values:

| Approach | Policies | Bindings | Maintenance Points | Audit Scope |
|---|---|---|---|---|
| **Without Templates** | 15 | 15 | 15 policies to update | 15 policies to review |
| **With Templates** | 1 | 15 | 1 policy to update | 1 template + bindings |

## When to Use Templates vs Static Policies

| Scenario | Recommendation |
|---|---|
| Multiple teams need identical permissions with different scopes | **Use template** |
| Single team with unique requirements | Static policy |
| Environment-based access (prod/staging/dev) | **Use template** |
| One-off admin or auditor policy | Static policy |
| Large-scale MZ migration (many MZs) | **Use template** (see **MZ2POL-08**) |

# 2. Template Syntax Deep Dive

## The `${bindParam:...}` Placeholder

Bind parameters are placeholders in the **WHERE condition** of a policy statement. They are replaced with actual values when the policy is bound to a group.

**Syntax:** `${bindParam:parameter-name}`

**Rules:**
- Parameters appear **only** in WHERE condition values — *"Parameters can be used in a policy similarly to how regular condition values are used."*
- In community practice, teams treat parameter names as case-sensitive and keep them to alphanumeric characters and hyphens — the templating page states no naming rules, so verify a new name with a test binding

## Single Parameter Example

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
```

When bound with `{"parameters": {"team": "checkout"}}`, this resolves to:

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout";
```

## Multiple Parameters Example

```
ALLOW storage:logs:read
  WHERE storage:dt.security_context = "${bindParam:team}"
  AND storage:bucket-name = "${bindParam:bucket}";
```

Binding: `{"parameters": {"team": "checkout", "bucket": "checkout_logs"}}`

## Multi-Statement Template

A single template can grant access across multiple data types using the same parameter:

```
ALLOW storage:buckets:read;
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:metrics:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW settings:objects:read WHERE settings:dt.security_context = "${bindParam:team}";
```

All four scoped statements resolve using the **same** `team` parameter value from the binding.

> **Every Grail read also needs `storage:buckets:read`.** It is *"Required additionally to a table permission"* — a policy that grants `storage:logs:read` alone returns no records. It is shown unconditional here because it does not take `dt.security_context`; it accepts only `storage:bucket-name`, `storage:table-name` and `storage:query-consumption`, so the record-level scope still comes from the table permissions below it. The templates in this notebook all include it.

### Where Parameters Can Be Used

The templating page does not publish a per-field support list. It says a parameter goes wherever a regular condition value goes, so the conditions each permission accepts — and the operators each condition supports — are the ones in the IAM policy statement reference. The fields this notebook parameterizes:

| Field | Domain | Operators to watch |
|---|---|---|
| `storage:dt.security_context` | Storage (logs, spans, metrics, events) | Use `MATCH` where the field can hold an array — `=`, `STARTSWITH` and `IN` on an array always return false |
| `storage:bucket-name` | Storage (bucket-level) | The only storage condition that supports negation |
| `settings:dt.security_context` | Settings objects | `=`, `IN`, `startsWith` |
| `settings:schemaId` | Settings schema | |
| `environment:management-zone` | Classic entity access | |

> **Sources:** Policy templating (DT docs) — *"Parameters can be used in a policy similarly to how regular condition values are used."*, IAM policy statements (DT docs) — `storage:buckets:read`: *"Required additionally to a table permission."*, Permissions in Grail (DT docs) — *"Using `=`, `STARTSWITH` or `IN` when the field holds an array will always return `false`."*

## 3. Creating a Parameterized Policy via UI

### Step-by-Step

1. Navigate to **Account Management** → **Identity & Access Management** → **Policies**
2. Click **Create policy**
3. Enter the policy name using the naming convention: `tpl-<scope>-<permission>`

4. Enter a description that mentions it is parameterized (e.g., "Parameterized: read access scoped to team security context")
5. Paste the parameterized policy statement with `${bindParam:...}` placeholders
6. Click **Save**

Dynatrace recognizes the bind parameters in the saved policy, but parameter **values** cannot be supplied from the group screens: *"Currently, group management does not support binding policies with parameters."* Create the binding through the Account Management API (Section 4) or Terraform (**IAM-95**).

### Naming Convention for Template Policies

| Convention | Example | Description |
|---|---|---|
| `tpl-team-data-reader` | Team-scoped read access | Read logs, spans, metrics by security context |
| `tpl-team-data-editor` | Team-scoped read+write | Full data access by security context |
| `tpl-env-data-reader` | Environment-scoped read | Read access by environment |
| `tpl-bucket-reader` | Bucket-scoped read | Read access by bucket name |
| `tpl-full-three-domain` | Full MZ replacement | All three domains scoped by parameter |

### Verification

After saving, the policy should appear in the policy list. Bind it with the API calls in Section 4 — group management in the UI cannot bind a parameterized policy.

> **Sources:** Policy templating (DT docs) — *"Currently, group management does not support binding policies with parameters."*

## 4. Binding with Parameters via IAM API

Binding a parameterized policy is an **API** operation — *"Currently, group management does not support binding policies with parameters"* — so the IAM API (or Terraform, **IAM-95**) is the only path, at any scale.

### API Endpoints

| Operation | Method | Endpoint |
|---|---|---|
| List policies | GET | `/iam/v1/repo/{levelType}/{levelId}/policies` |

| Operation | Method | Endpoint |
| --- | --- | --- |
| List bindings | GET | `/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}` |
| Create binding | POST | `/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}/{groupUuid}` |
| Delete binding | DELETE | `/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}/{groupUuid}` |

**Authentication:** OAuth 2.0 bearer token issued to an OAuth client with IAM management scopes. The Account Management API documents OAuth tokens only — classic API tokens are not an option.

## Step 1: Find Policy UUID

```
curl -X GET \
  'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/policies' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

Look for your template policy name (e.g., `tpl-team-data-reader`) in the response and note its `uuid`.

## Step 2: Find Group UUID

```
curl -X GET \
  'https://api.dynatrace.com/iam/v1/accounts/&lt;ACCOUNT_UUID&gt;/groups' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

## Step 3: Create Binding with Parameters

```
curl -X POST \

'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/bindings/&lt;POLICY_UU
ID&gt;/&lt;GROUP_UUID&gt;' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;' \
  -H 'Content-Type: application/json' \
  -d '{"parameters": {"team": "checkout"}}'
```

## Step 4: Verify Binding

```
curl -X GET \

'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/bindings/&lt;POLICY_UU
ID&gt;' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

### Bulk Binding: Multiple Teams

Bind the same template to multiple groups with different parameter values:

```
#!/bin/bash
ACCOUNT_UUID="&lt;your-account-uuid&gt;"
POLICY_UUID="&lt;team-reader-template-uuid&gt;"
TOKEN="&lt;bearer-token&gt;"
API_BASE="https://api.dynatrace.com/iam/v1/repo/account/${ACCOUNT_UUID}"

declare -A TEAM_GROUPS=(
  ["checkout"]="&lt;group-uuid-checkout&gt;"
  ["payments"]="&lt;group-uuid-payments&gt;"
  ["catalog"]="&lt;group-uuid-catalog&gt;"
  ["search"]="&lt;group-uuid-search&gt;"
  ["frontend"]="&lt;group-uuid-frontend&gt;"
)

for team in "${!TEAM_GROUPS[@]}"; do
  GROUP_UUID="${TEAM_GROUPS[$team]}"
  echo "Binding template to ${team} (${GROUP_UUID})..."
  curl -s -X POST \
    "${API_BASE}/bindings/${POLICY_UUID}/${GROUP_UUID}" \
    -H "Authorization: Bearer ${TOKEN}" \
    -H 'Content-Type: application/json' \
    -d "{\"parameters\": {\"team\": \"${team}\"}}" \
    -o /dev/null -w "  HTTP %{http_code}  (204 = bound; 400 'already exists' = already
converged)\n"
done
```

> **Tip:** Success is **HTTP 204** with an empty body. Re-posting an identical binding returns **400** "such binding already exists" — treat it as converged, not as a failure (**IAM-96** Step 7, live-verified).
>
> **Full working example:** For a complete script that creates groups, policies across all three domains (UI + Data + Config), and binds everything together for a four-persona model, see the **End-to-End Provisioning Script** in **IAM-11: Policy Persona Workshop**.

## 5. Monaco YAML for Policy-as-Code

Version-controlled, peer-reviewed, CI/CD-deployed IAM configuration.

### What Monaco Can and Cannot Express

Monaco manages IAM as **account resources** — `policies:`, `groups:` and `boundaries:` are dedicated top-level YAML types (not Settings schemas), deployed with Monaco's dedicated account commands rather than `monaco deploy`. Its documented account format covers policies, groups, boundaries and the policy-to-group binding with boundaries — but **not binding** **`parameters`**. Bind

templated ( `${bindParam:…}` ) policies with their parameter values through the IAM API (Section 4) or Terraform (**IAM-95**); use Monaco for the policy text itself and for unparameterized bindings.

## Project Structure

```
iam-config/
├── manifest.yaml          # includes an accounts: section with the OAuth client
└── account/
    └── iam.yaml           # policies:, boundaries:, groups:
```

## Policy Template YAML

```
# account/iam.yaml
policies:
  - name: tpl-team-data-reader
    id: tpl-team-data-reader
    level:
      type: account
    description: "Parameterized: read access scoped to ${bindParam:team}"
    policy: |-
      ALLOW storage:buckets:read;
      ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
      ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}";
      ALLOW storage:metrics:read WHERE storage:dt.security_context = "${bindParam:team}";
```

The bindings that give `team` its value — `checkout` for one group, `payments` for another — are then created with the IAM API calls in Section 4 or the `dynatrace_iam_policy_bindings_v2` resource in **IAM-95**, both of which accept `parameters`.

> **Sources:** Monaco account configuration (DT docs) — *"Using Monaco, you can define users, service users, groups, policies, and boundaries as dedicated types in YAML configuration files."* and *"existing commands like `monaco deploy` ignore any account configuration that may be defined in a manifest file."*, Monaco commands (DT docs) — `account` with `deploy` : *"Deploy Account Management resources."*

## GitOps Workflow

1. **Branch** — Create feature branch for IAM change
2. **Edit** — Modify policy or binding YAML
3. **Review** — Submit PR, get peer review from IAM admin
4. **Test** — Deploy to a test account with `monaco account deploy` — *"existing commands like `monaco deploy` ignore any account configuration"*
5. **Approve** — Merge after validation
6. **Deploy** — CI/CD applies to production

See **AUTOM-03: Monaco** for the full automation workflow.

# 6. Common Template Patterns

## Pattern 1: Team-Scoped Data Access

The most common pattern — scope all data access by team security context.

```
ALLOW storage:buckets:read;
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:metrics:read WHERE storage:dt.security_context = "${bindParam:team}";
```

**Binding:** `{"parameters": {"team": "checkout"}}` or `{"parameters": {"team": "payments"}}`

## Pattern 2: Environment-Scoped Access

Scope access by environment (prod, staging, dev).

```
ALLOW storage:buckets:read;
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read
  WHERE storage:dt.security_context = "${bindParam:environment}";
ALLOW settings:objects:read WHERE settings:dt.security_context =
"${bindParam:environment}";
```

**Binding:** `{"parameters": {"environment": "prod"}}` or `{"parameters": {"environment": "staging"}}`

## Pattern 3: Bucket-Scoped Team Isolation

Hard data isolation using dedicated buckets per team.

```
ALLOW storage:buckets:read WHERE storage:bucket-name = "${bindParam:log-bucket}";
ALLOW storage:buckets:read WHERE storage:bucket-name = "${bindParam:span-bucket}";
ALLOW storage:buckets:read WHERE storage:bucket-name = "${bindParam:metric-bucket}";
ALLOW storage:logs:read WHERE storage:bucket-name = "${bindParam:log-bucket}";
ALLOW storage:spans:read WHERE storage:bucket-name = "${bindParam:span-bucket}";
ALLOW storage:metrics:read WHERE storage:bucket-name = "${bindParam:metric-bucket}";
```

**Binding:** `{"parameters": {"log-bucket": "checkout_logs", "span-bucket": "checkout_spans", "metric-bucket": "checkout_metrics"}}`

## Pattern 4: Combined Team + Environment

Defense-in-depth: require both team and bucket match.

```
ALLOW storage:buckets:read WHERE storage:bucket-name = "${bindParam:bucket}";
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read
  WHERE storage:dt.security_context = "${bindParam:team}"
  AND storage:bucket-name = "${bindParam:bucket}";
```

**Binding:** `{"parameters": {"team": "checkout", "bucket": "prod_checkout_logs"}}`

## Pattern 5: Settings Schema Scoped

Grant access to specific settings categories.

```
ALLOW settings:objects:read, settings:objects:write
  WHERE settings:schemaId startsWith "${bindParam:schema-prefix}";
```

**Binding:** `{"parameters": {"schema-prefix": "builtin:openpipeline.logs."}}` for a log-processing team

`{"parameters": {"schema-prefix": "builtin:alerting"}}` works only on **Dynatrace Classic**: alerting profiles, maintenance windows and problem notifications are not visible in Latest Dynatrace. There, an alerting team needs `automation:workflows:*` for notifications and `builtin:davis.anomaly-detectors` for custom alerts (IAM-04 Pattern 6).

> **Sources:** Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — *"None of the schemas on this page are visible in Latest Dynatrace."*

## Pattern 6: Full Three-Domain (MZ Replacement)

Complete replacement of a Management Zone — covers entity, data, and settings access.

```
ALLOW storage:buckets:read;
ALLOW storage:entities:read WHERE storage:dt.security_context = "${bindParam:scope}";
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read
  WHERE storage:dt.security_context = "${bindParam:scope}";
ALLOW settings:objects:read, settings:objects:write
  WHERE settings:dt.security_context = "${bindParam:scope}";
```

> **Storage writes cannot be condition-scoped (live-verified 07/2026)** — `storage:*` wildcards are rejected, and `storage:logs:write WHERE ...` fails with `Invalid condition name`. Grant `storage:logs:write, storage:metrics:write, storage:events:write` unscoped in a separate policy only where genuinely needed.
>
> **`storage:entities:read` takes no `MATCH`.** Its conditions are `storage:entity.type` and `storage:dt.security_context`, each with `=`, `IN` and `startsWith` only — and *"entity*

> *permissions only allow filtering for the* `dt.security_context` *field."* Because `=` on an array-valued field always returns false, confirm the entities you need carry a single-valued security context (the coverage queries in Section 7) before relying on this line.

For classic entity-level restriction through management zones, combine it with a **boundary** using the same scope.

> **Sources:** IAM policy statements (DT docs), Permissions in Grail (DT docs) — *"In contrast to monitoring data, entity permissions only allow filtering for the* `dt.security_context` *field."* and *"Using* `=`, `STARTSWITH` *or* `IN` *when the field holds an array will always return* `false`*."*

**Binding:** `{"parameters": {"scope": "LOB5"}}` or `{"parameters": {"scope": "team-frontend"}}`

> **For MZ migration at scale**, see **MZ2POL-08: Templated Policies for MZ Migration** which covers converting MZ patterns to these templates.

## Pattern Summary

| Pattern | Parameters | Use Case |
|---|---|---|
| Team-Scoped | `team` | Team data isolation via security context |
| Environment-Scoped | `environment` | Environment-based access control |
| Bucket-Scoped | `log-bucket`, `span-bucket`, `metric-bucket` | Bucket-level isolation for retention, cost or compliance — buckets are *"not intended as the primary mechanism for securing or segmenting data access"* |
| Combined | `team`, `bucket` | Defense-in-depth (team + bucket) |
| Settings Schema | `schema-prefix` | Capability-based settings access |
| Three-Domain | `scope` | Full MZ replacement |

> **Sources:** Partition data (DT docs) — *"They are not intended as the primary mechanism for securing or segmenting data access."*

## 7. DQL Audit Queries

Use these queries to verify that templated policy bindings are working correctly and to audit policy-related API activity.

> **Group, policy, boundary and binding changes are account-level, and this environment's audit events are not their change log.** *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings" — "Administrators can view these logs in Account Management > Settings > Audit log"*, where they are kept for up to ten years (also queryable through the Account Audits API). The same page says *"The Account Management audit log doesn't yet cover every account-level change. Some IAM configuration and login events aren't recorded."* The query below shows IAM API calls made **through this environment** — useful for spotting automation, not a change log. These events show IAM API calls made through this environment, including environment-level policy-binding changes; scheduled `/lookups/iam_*` file uploads are excluded by the query. Account-level group and policy changes are recorded in the account audit log.
>
> **Sources:** Account Management audit logs (DT docs).

```
// IAM API activity in this environment (binding changes made in Account Management:
account audit log)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell
used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail
tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit
records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days
here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is
replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE", "UPDATE"}) and
contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// Find access denied events that may indicate parameter misconfiguration
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type (GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path), origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.outcome == "403"
| summarize denials = count(), by:{resource, user.id}
| sort denials desc
| limit 25
```

```
// Verify security context coverage — entities need context for template binding to work
fetch dt.entity.service
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missingContext = countIf(isNull(dt.security_context))
| fields total, withContext, missingContext,
         coveragePercent = round(100.0 * withContext / total, decimals: 2)

// Smartscape note (dt.entity.* is deprecated but still functional): dt.security_context exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so isNull / isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would miscount coverage.
// Keep the classic query above.
```

```
// Verify security context values match expected team parameters
fetch dt.entity.service
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc

// Smartscape note (dt.entity.* is deprecated but still functional): dt.security_context
exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so isNull /
isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would miscount
coverage.
// Keep the classic query above.
```

# 8. Troubleshooting Templated Policies

## Common Issues

| Issue | Symptom | Cause | Resolution |
|---|---|---|---|
| Parameter not resolved | Access denied despite binding | Parameter name mismatch (case-sensitive) | Verify parameter name matches exactly between policy and binding |
| Binding not applied | No access at all | Missing binding POST call | Verify binding exists via GET bindings endpoint |
| Wrong scope | User sees too much or too little | Parameter value incorrect | Check binding parameter values via API |
| Template edit rejected | Saving a bound policy fails | The edit adds, removes or renames a parameter | *"Modification of bound policies is allowed only if the parameter set doesn't change."* Keep the parameter names, or create a new template and re-bind |
| One group needs several scopes | Separate bindings per scope pile up | Each binding carries one value per parameter | Use a **list parameter**: write the condition with `IN ("${bindParam:teams}")` and bind *"a comma-separated list within a single string value"* — e.g. `{"parameters": {"teams": "checkout,payments"}}` |
| Multiple parameters misaligned | Partial access | One parameter correct, another wrong | Check all parameter values in binding response |

### Debugging Checklist

1. **Verify policy exists** and contains `${bindParam:...}` syntax
2. **Verify group UUID** is correct (list groups via API)
3. **Verify binding exists** (GET bindings endpoint — check response includes your group)
4. **Verify parameter values** in the binding response match expected values
5. **Verify security context** on target entities matches the parameter value
6. **Test with admin account** to confirm the data exists
7. **Check audit logs** for denied events (use DQL queries above)

> **Sources:** Policy templating (DT docs) — *"Modification of bound policies is allowed only if the parameter set doesn't change"*; list values are assigned by *"providing a comma-separated list within a single string value."*
>
> **Tip:** Use the GET bindings endpoint to dump all bindings for a policy and programmatically compare parameter values against expected values. This catches typos and mismatches quickly.

For comprehensive IAM troubleshooting, see **IAM-09: Troubleshooting Access Issues**.

---

## Summary

In this notebook, you learned:

- Why policy templates reduce policy sprawl and simplify management
- The `${bindParam:parameter-name}` syntax and where it can be used
- How to create parameterized policies via the Dynatrace UI
- How to bind policies with parameters using the IAM API (including bulk operations)
- Monaco YAML patterns for policy-as-code with templates
- Six common template patterns for team, environment, and bucket scoping
- DQL audit queries to verify bindings are working
- Troubleshooting methodology for parameterized policies

## Next Steps

- **IAM-05: Boundary Design Patterns** — Combine templated policies with boundaries for defense in depth
- **IAM-07: Audit Logging and Compliance** — Set up ongoing audit monitoring for policy bindings
- **AUTOM-03: Monaco** — Full Monaco configuration automation workflow
- **MZ2POL-08: Templated Policies for MZ Migration** — Apply templates to MZ migration at scale

- **IAM-11: Policy Persona Workshop** — Map organizational personas to policy templates using a structured design methodology. Design your persona model, then use **IAM-12** for the provisioning scripts that creates groups, policies (UI + Data + Config), and templated bindings for a four-persona model via the IAM API.

---

## References


- Policy Templating
- Policy Management API
- Working with Policies
- IAM Policy Statement Syntax


---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*